Chapter V

YARN
Yet Another Resource Negotiator

1. Introduction & Motivation
2. YARN Architecture Overview
3. Application Workflow in Hadoop YARN
4. Scheduling & Resource Allocation
5. YARN Administration

Objectives

1. Describe the architecture and key components of YARN (ResourceManager, NodeManager, ApplicationMaster, and Containers) and their interactions
2. Explain how YARN decouples resource management from application logic. Enabling the execution of diverse processing models including MapReduce.
3. Analyze the complete lifecycle of a MapReduce job on YARN from submission to execution and resource cleanup.

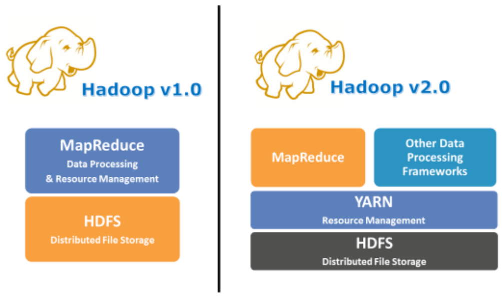
Section N° 1

Introduction & Motivation

What is YARN

- YARN(Yet Another Resource Negotiator) is cluster resource management system for Hadoop
- YARN was introduced in Hadoop 2 to improve the MapReduce implementation and eliminate Hadoop 1 scheduling disadvantages
- YARN is a connecting link between high level applications(Spark, HBase etc.)and low-level Hadoop environment
- With introduction of YARN, Hadoop transformed from only MapReduce framework to big data processing core
- Some people characterized YARN as a large-scale, distributed operating system for big data applications.

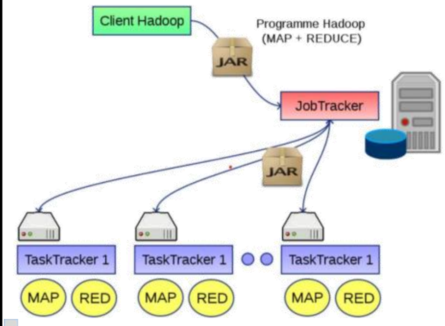
Why do we need YARN?



The primary goal of YARN is to decouple the functionalities of resource management and data processing into separate components.

This allows a global resource manager that can support diverse data processing applications, such as MapReduce, Spark, Storm, Tez, and others, to run on a single cluster.

Problems With Hadoop 1



Job Tracker

- Part of the Master node.
- The responsibility of job tracker is to send the user task to task tracker where task tracker applies MapReduce functions to perform that task with data stored in data nodes.
- It is also Job tracker responsibility to track your task, and if it's not completed successfully somehow to redirect it to some other task tracker.

Task Tracker

- Part of slave nodes.
- Each slave node in Hadoop 1 has task tracker with data node.
- Its responsibility is to run task/jobs given by job tracker.
- Task tracker run with data node, as each commodity hardware contain one instance of the data node and task tracker.
- Job Tracker maintains the locations of task tracker as task tracker and job tracker can communicate with each other.
- Job tracker exactly knows where task tracker exists on which hardware.

Limitations of MapReduce 1

Limitation

Scalability Bottleneck of JobTracker

The JobTracker can efficiently manage up to 4,000 nodes and around 40,000 tasks concurrently.
Beyond this, latency increases, job scheduling slows down, and failures become more frequent.

Static Resource Allocation Wastes Resources

Each map or reduce task is given a fixed amount of memory, often around 1 GB per task.
If a task only needs 300 MB, 700 MB is wasted—multiplied across thousands of tasks.

Single Application Type (MapReduce Only)

100% of the cluster's compute resources are dedicated to MapReduce jobs.
Other applications (like Spark, HBase batch jobs) could not run, causing underutilization in mixed workload scenarios.

Job Failures and Recovery Issues

Node failure rate: ~2% per day in a 2,000-node cluster.
MRv1 re-runs failed tasks but has no efficient fault-tolerant scheduling, leading to higher job completion times.

Poor Multi-tenancy

No queue-level isolation or priority management.
In shared clusters, jobs from one user could monopolize all resources.

Design Goals for YARN

hadoop YARN

Scalability

YARN is designed to perform efficiently on a network of a myriad of nodes.

Support for Programming Model

MapReduce is no doubt great for many applications, but not for everything in the world of computation.

High cluster utilization

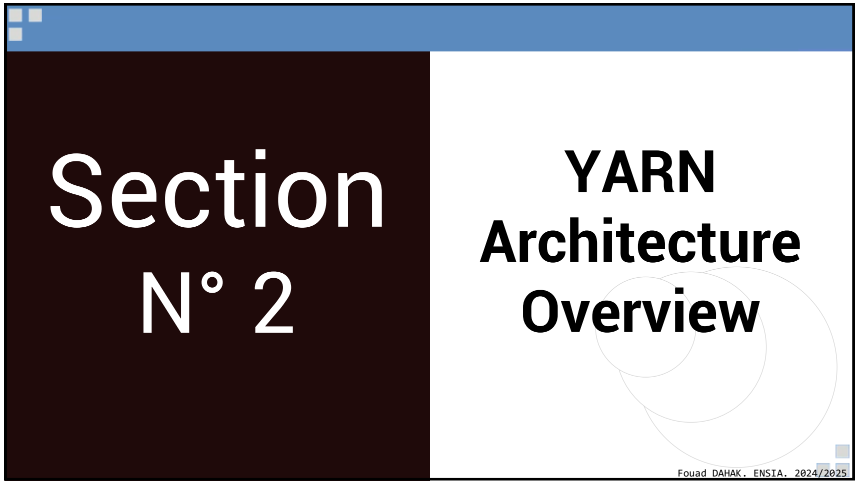
YARN fine-graine resources in terms of RAM, CPU, and disk (containers), leading to an optimal utilization of the available resources.

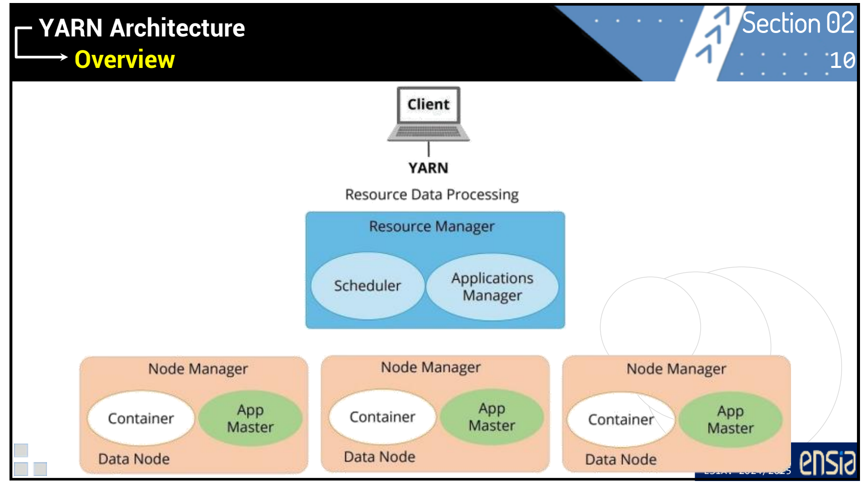
Multitenancy

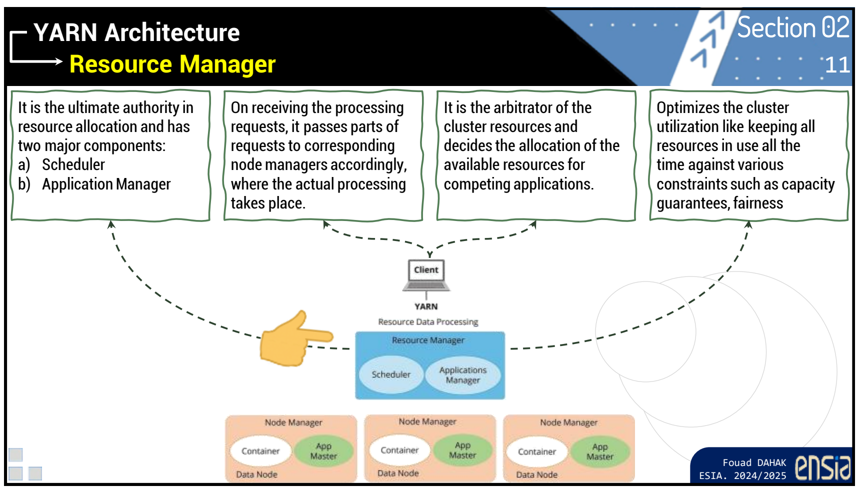
YARN has the notion of pluggable schedulers and the Capacity Scheduler with YARN has been enhanced to provide a flexible resource model that enables multiple tenants to securely share the cluster in an optimized way

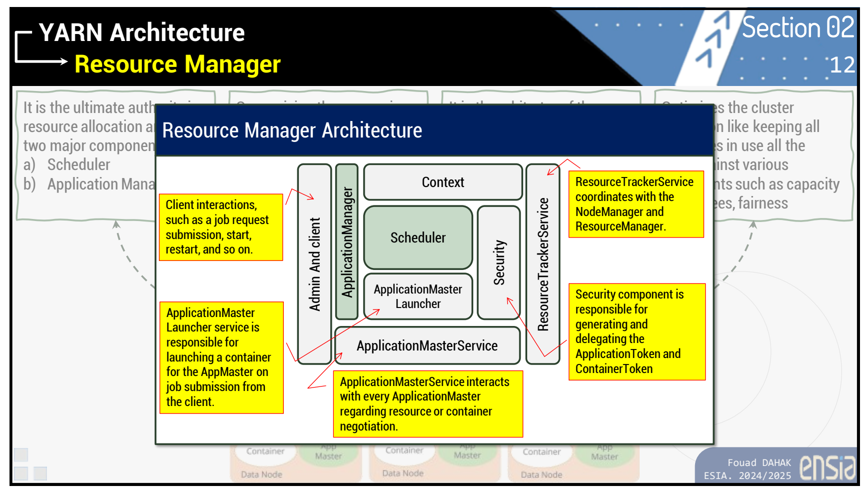
Locality Awareness

This is a key requirement for YARN when dealing with big data; moving computation is cheaper than moving data.
This helps to minimize network congestion and increase the overall throughput of the system


Section
N° 2
YARN
Architecture
Overview
Fouad DAHAK. ENSIA. 2024/2025


YARN Architecture
Overview
Section 02
10
Client
YARN
Resource Data Processing
Resource Manager
Scheduler
Applications Manager
Node Manager
Container
App Master
Data Node
Node Manager
Container
App Master
Data Node
Node Manager
Container
App Master
Data Node
ensia


YARN Architecture
Resource Manager
Section 02
11
It is the ultimate authority in resource allocation and has two major components:
a) Scheduler
b) Application Manager
On receiving the processing requests, it passes parts of requests to corresponding node managers accordingly, where the actual processing takes place.
It is the arbitrator of the cluster resources and decides the allocation of the available resources for competing applications.
Optimizes the cluster utilization like keeping all resources in use all the time against various constraints such as capacity guarantees, fairness
Client
YARN
Resource Data Processing
Resource Manager
Scheduler
Applications Manager
Node Manager
Container
App Master
Data Node
Node Manager
Container
App Master
Data Node
Node Manager
Container
App Master
Data Node
Fouad DAHAK
ESIA. 2024/2025
ensia


YARN Architecture
Resource Manager
Section 02
12
Resource Manager Architecture
Client interactions, such as a job request submission, start, restart, and so on.
ApplicationMaster Launcher service is responsible for launching a container for the AppMaster on job submission from the client.
Admin And client
ApplicationManager
Context
Scheduler
ApplicationMaster Launcher
Security
ApplicationMasterService
ResourceTrackerService
ApplicationMasterService interacts with every ApplicationMaster regarding resource or container negotiation.
ResourceTrackerService coordinates with the NodeManager and ResourceManager.
Security component is responsible for generating and delegating the ApplicationToken and ContainerToken
Fouad DAHAK
ESIA. 2024/2025
ensia

YARN Architecture
Resource Manager: Scheduler

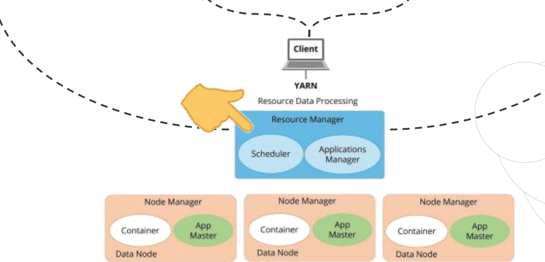
Section 02
13

Responsible for allocating resources to the various running applications.

Performs scheduling based on the resource requirements of the applications.

It has a pluggable policy plug-in, which is responsible for partitioning the cluster resources among the various applications.

There are two such plug-ins:
- Capacity Scheduler
- Fair Scheduler.



Fouad DAHAK
ESIA. 2024/2025 ensia

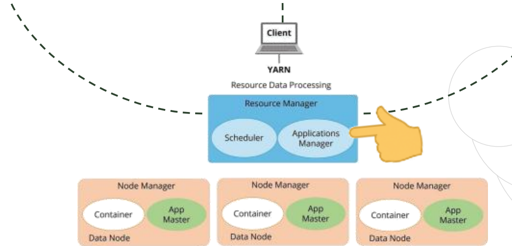
YARN Architecture
Resource Manager: Application Manager

Section 02
14

Responsible for accepting job submissions.

Negotiates the first container from the Resource Manager for executing the application specific Application Master.

Manages running the Application Masters in a cluster and provides service for restarting the Application Master container on failure.



Fouad DAHAK
ESIA. 2024/2025 ensia

YARN Architecture
Node Manager

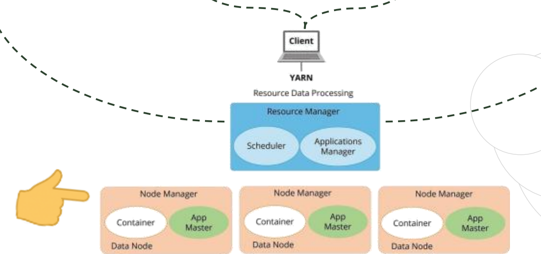
Section 02
15

Takes care of individual nodes in a Hadoop cluster and manages user jobs and workflow on the given node.

Registers with the Resource Manager and sends heartbeats with the health status of the node.

Manage application containers assigned to it by the resource manager.

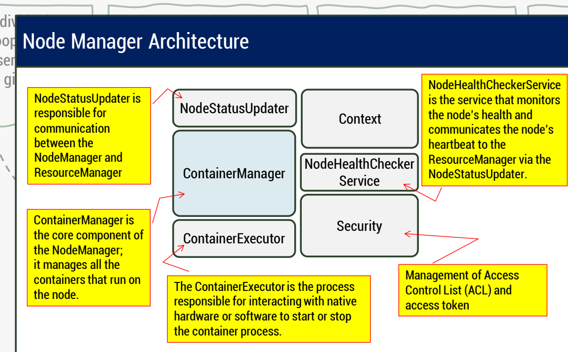
Creates the requested container process and starts it. Monitors resource usage (memory, CPU) of individual containers.



Fouad DAHAK
ESIA. 2024/2025 ensia

YARN Architecture
Node Manager

Section 02
16



Fouad DAHAK
ESIA. 2024/2025 ensia

YARN Architecture
Node Manager: Application Master

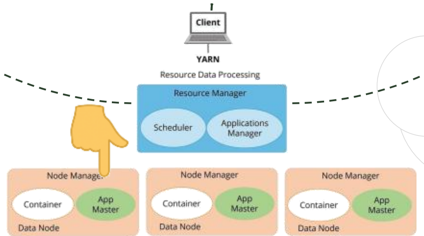
Section 02
17

Each application has a unique Application Master associated with it which is a framework specific entity.

It is the process that coordinates an application's execution in the cluster and also manages faults.

It negotiates resources from the Resource Manager and work with the Node Manager to execute and monitor the component tasks.

Once started, it periodically sends heartbeats to the Resource Manager to affirm its health and to update its resource demands.



An application is a single job submitted to the framework

Fouad DAHAK
ESIA. 2024/2025

YARN Architecture
Node Manager: Container

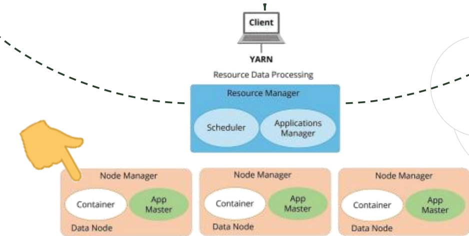
Section 02
18

It is a collection of physical resources such as RAM, CPU cores, and disks on a single node.

Containers are managed by a container launch context which is container life-cycle(CLC).

CLC contains a map of environment variables, dependencies stored in a remotely accessible storage.

It grants rights to an application to use a specific amount of resources (memory, CPU etc.) on a specific host.



Fouad DAHAK
ESIA. 2024/2025

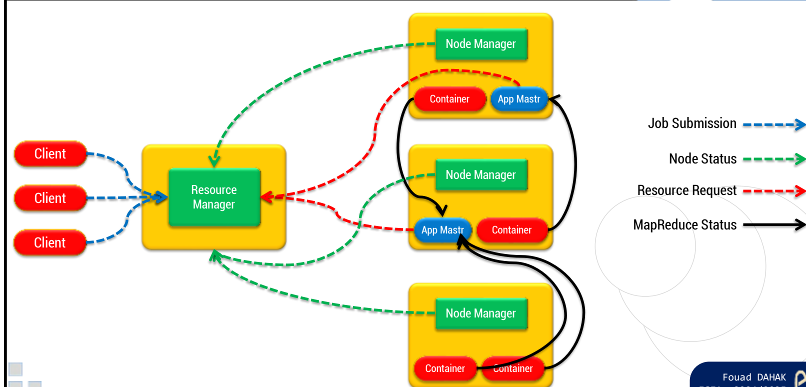
Section N° 3

Application Workflow in Hadoop YARN

Fouad DAHAK. ENSIA. 2024/2025

Application Workflow in Hadoop YARN
Overview

Section 03
20



Fouad DAHAK
ESIA. 2024/2025

Application Workflow in Hadoop YARN
Workflow

Section 03 21

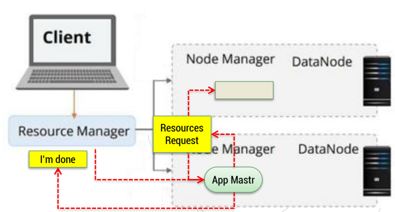
Step 1: Client submits an application to the Resource Manager
Users submit applications to the Resource Manager by typing the Hadoop jar command. The Resource Manager maintains the list of applications on the cluster and available resources on the Node Manager.

Step 2 - Resource Manager allocates Container
The Scheduler selects a container. Then, the Application Master is started and is responsible for the entire life-cycle of that particular application. First, it sends resource requests to the ResourceManager to ask for containers to run the application's tasks.

Step 3 - Application Master contacts Node Manager
After a container is allocated, the Application Master asks the Node Manager managing the host on which the container was allocated to use these resources to launch an application-specific task.

Step 4 -Resource Manager Launches Container
Throughout its life, the Application Master negotiates containers to launch all of the tasks needed to complete its application. It also monitors the progress of an application and its tasks, restarts failed tasks, and reports progress back to the client

Step 5 - Container Executes the Application Master
After the application is complete, the Application Master shuts itself and releases its own container.

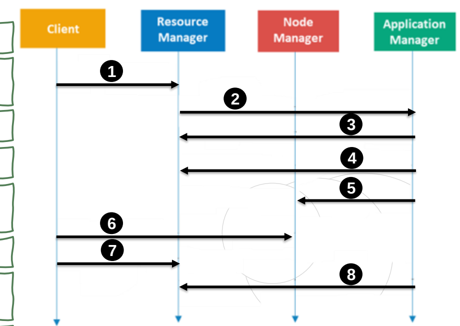


Fouad DAHAK ESIA. 2024/2025 ensia

Application Workflow in Hadoop YARN
Workflow

Section 03 22

1. Client submits an application
2. Resource Manager allocates a container to start Application Manager
3. Application Manager registers with Resource Manager
4. Application Manager asks containers from Resource Manager
5. Application Manager notifies Node Manager to launch containers
6. Application code is executed in the container
7. Client contacts Resource Manager to monitor application's status
8. Application Manager unregisters with Resource Manager



Fouad DAHAK ESIA. 2024/2025 ensia

QUIZ

1. What does YARN stand for?
A) Yet Another Random Name
B) Yet Another Resource Negotiator
C) Your Assigned Resource Node
D) Yield And Run Node

2. What component of YARN is responsible for managing resources in the cluster?
A) NodeManager
B) ApplicationMaster
C) ResourceManager
D) JobTracker

3. What is the role of the NodeManager in YARN?
A) Scheduling all cluster resources
B) Managing the local node's containers and reporting status to ResourceManager
C) Launching MapReduce jobs
D) Storing HDFS blocks

4. Which component is created for every application submitted in YARN?
A) ResourceManager
B) NameNode
C) ApplicationMaster
D) JobTracker

5. What is a container in YARN?
A) A file archive used to deploy applications
B) A unit of resource allocation that contains CPU, memory, etc.
C) A scheduler that controls job priorities
D) A user account in Hadoop

6. What happens if the ApplicationMaster fails during execution?
A) The entire job is aborted
B) YARN restarts the ApplicationMaster up to a configured retry limit
C) Nothing; jobs continue normally
D) All nodes are restarted

7. Which YARN scheduler supports resource fairness among multiple users?
A) FIFO Scheduler
B) Round Robin Scheduler
C) Capacity Scheduler
D) Local Scheduler

8. Which YARN component is responsible for launching containers on behalf of apps?
A) ApplicationMaster
B) NodeManager
C) ResourceManager
D) DataNode

9. What is the relationship between YARN and MapReduce?
A) YARN replaces MapReduce
B) YARN runs on top of MapReduce
C) MapReduce runs on top of YARN as a framework
D) YARN is a plug-in for MapReduce

10. Which of the following best describes the role of ResourceManager in YARN?
A) Distributes HDFS blocks to nodes
B) Assigns containers and monitors their execution
C) Manages the cluster and decides where containers should run
D) Handles the replication of HDFS blocks

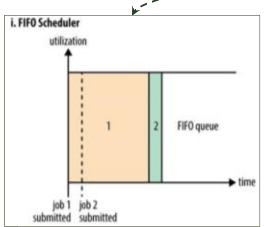
Section N° 4

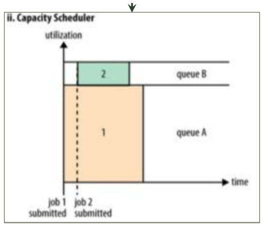
Scheduling & Resource Allocation

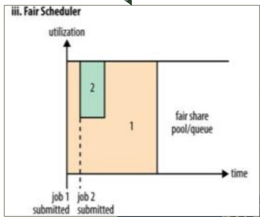
Fouad DAHAK. ENSIA. 2024/2025

Scheduling & Resource Allocation
Scheduling in YARN

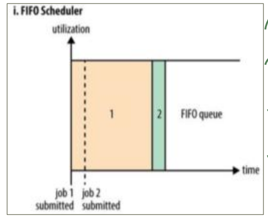
The ResourceManager acts as a pluggable global scheduler that manages and controls all the containers (resources). There are three different policies that can be applied over the scheduler, as per requirements and resource availability.







Scheduling & Resource Allocation
Scheduling in YARN: FIFO



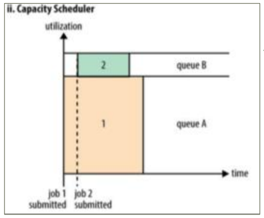
The FIFO Scheduler places applications in a queue and runs them in the order of submission (first in, first out).

Requests for the first application in the queue are allocated first; once its requests have been satisfied, the next application in the queue is served, and so on..

The FIFO Scheduler has the merit of being simple to understand and not needing any configuration, but it's not suitable for shared clusters.

Large applications will use all the resources in a cluster, so each application has to wait for its turn. On a shared cluster it is better to use the Capacity Scheduler or the Fair Scheduler. Both of these allow long running jobs to complete in a timely manner.

Scheduling & Resource Allocation
Scheduling in YARN: Capacity Scheduler



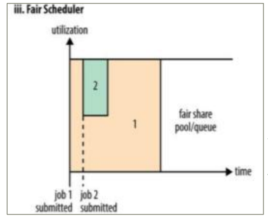
The Capacity Scheduler is designed to run Hadoop Map Reduce as a shared, multi-tenant cluster in an operator-friendly manner while maximizing the throughput and the utilization of the cluster while running Map Reduce applications.

The Capacity Scheduler is designed to allow sharing a large cluster while giving each organization a minimum capacity guarantee.

The central idea is that the available resources in the Hadoop Map Reduce cluster are partitioned among multiple organizations who collectively fund the cluster based on computing needs.

There is an added benefit that an organization can access any excess capacity not being used by others. This provides elasticity for the organizations in a cost-effective manner.

Scheduling & Resource Allocation
Scheduling in YARN: Fair Scheduler



Fair scheduling is a method of assigning resources to jobs such that all jobs get, on average, an equal share of resources over time.

When there is a single job running, that job uses the entire cluster. When other jobs are submitted, tasks slots that free up are assigned to the new jobs, so that each job gets roughly the same amount of CPU time.

The fair scheduler organizes jobs into pools, and divides resources fairly between these pools. By default, there is a separate pool for each user, so that each user gets an equal share of the cluster.

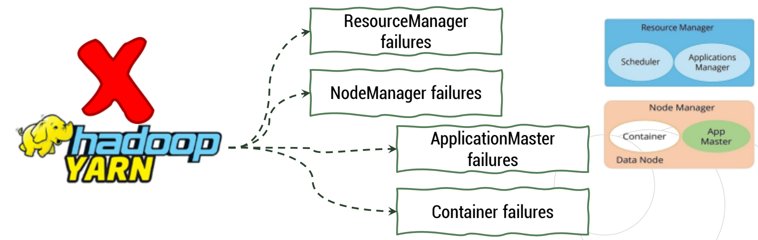
It is also possible to set a job's pool based on the user's Unix group or any jobconf property. Within each pool, jobs can be scheduled using either fair sharing or first-in-first-out (FIFO) scheduling.

If a pool's minimum share is not met for some period of time, the scheduler optionally supports preemption of jobs in other pools. The pool will be allowed to kill tasks from other pools to make room to run.

Section N° 5

Yarn Administration

Yarn Administration
Failures in YARN

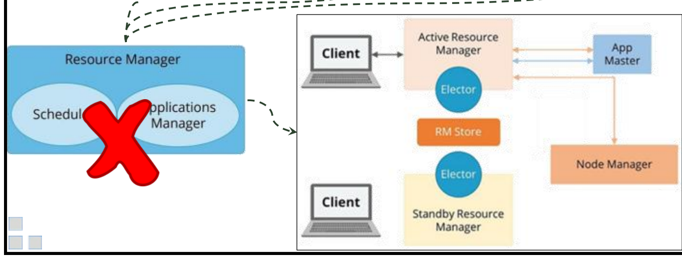


Yarn Administration
Failures in YARN: ResourceManager failures

The ResourceManager stores the state of the cluster, such as the metadata of the submitted application, information on cluster resource containers, information on the cluster's configurations...

If the ResourceManager goes down because of some hardware failure, then there is no way to avoid manually debugging the cluster and restarting the ResourceManager.

During the time the it is down, the cluster is unavailable, and once it gets restarted, all jobs would need a restart, so the half-completed jobs lose any data and need to be restarted again.



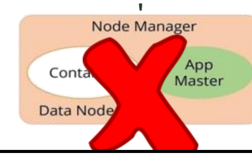
Yarn Administration
Failures in YARN: NodeManager failures

If a Node Manager fails, the ResourceManager detects this failure using a time-out (that is, stops receiving the heartbeats from the NodeManager).

The ResourceManager removes the NodeManager from its pool of available NodeManagers. It also kills all the containers running on that node & reports the failure to all running AMs.

AMs are then responsible for reacting to node failures, by redoing the work done by any containers running on that node during the fault.

If the fault is transient then the Node Manager will resynchronizes with the ResourceManager. On the similar lines if a new Node Manager joins the cluster, the RM notifies all AMs about the availability of new resources.



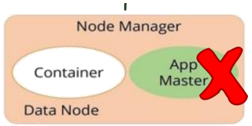
Yarn Administration
Failures in YARN: ApplicationMaster failures

To recover the application's state after its restart because of an ApplicationMaster failure is the responsibility of the ApplicationMaster itself.

When the ApplicationMaster fails, the ResourceManager simply starts another container with a new ApplicationMaster running in it for another application attempt.

It is the responsibility of the new ApplicationMaster to recover the state of the older one, and this is possible only when AppMasters persist their states in the external location so that it can be used for future reference.

Any ApplicationMaster can run any application from scratch instead of recovering its state and rerunning again.



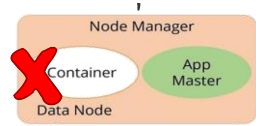
Yarn Administration
Failures in YARN: Container failures

When a container finishes, the ResourceManager notifies the AM, which then interprets the container's exit status as success or failure.

The AM handles job container failures. The application framework is responsible for managing these failures, while YARN provides the necessary information.

As part of processing the API response, the ResourceManager gathers information on finished containers from the AM, which receives this data directly from the containers.

To handle container allocation failures, the ResourceManager uses the Allocate call to collect container info. The Allocate Response often returns no containers, but periodic calls ensure allocation. When a container is assigned, resources are guaranteed, and the AM won't get more containers than requested.



Yarn Administration
Development And Administration Tools

These tools enable developers to submit, monitor, and manage jobs on the YARN cluster.

YARN Web UI

YARN web UI runs on 8088 port by default. It also provides a better view than Hue; however, you cannot control or configure from YARN web UI.

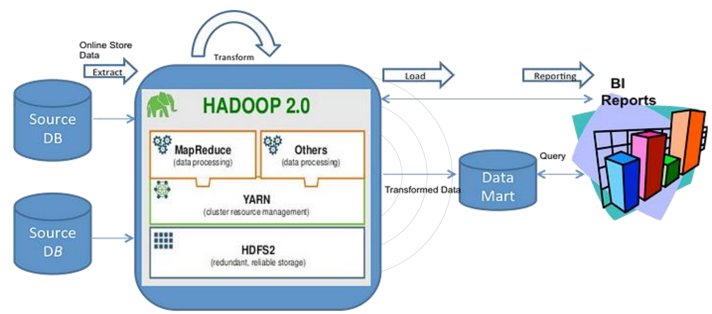
Hue Job Browser

Hue Job Browser The Hue Job Browser allows you to monitor the status of a job, kill a running job, and view logs.

YARN Command Line

Most of the YARN commands are for the administrator rather than the developer.

Conclusion
The Big Picture



The Big Data ETL (exploded)


Chapter V : YARN
Yet Another Resource Negotiator
37
Thank
You
Q & A
Fouad DAHAK
ESIA. 2024/2025
ensia